

基于多种人工智能方法的 2048 游戏智能体设计与实现报告



团队：

王清香 (1820241084) | 1820241084@bit.edu.cn | 13261682353

薛学刚 (1820241085) | 1820241085@bit.edu.cn | 13120370010

洪维斌 (1820241086) | 1820241086@bit.edu.cn | 13146789122

07842401

人工智能概论大作业

目录

摘要.....	4
1. 引言.....	6
1.1 研究背景.....	6
1.2 研究目的与意义.....	6
1.3 技术路线.....	7
1.4 团队分工.....	8
1.5 报告结构.....	9
2. 问题定义与系统总体方案.....	11
2.1 任务定义.....	11
2.2 问题建模.....	11
2.3 系统总体方案.....	13
2.4 系统流程.....	14
2.5 本章小结.....	15
3. 系统设计与关键方法.....	16
3.1 游戏环境设计.....	16
3.2 状态表示与动作空间设计.....	19
3.3 启发式搜索策略设计.....	20
3.4 进化算法优化设计.....	22
3.5 监督学习模型设计.....	24
3.6 强化学习模型设计.....	25
3.7 本章小结.....	27
4. 模型训练与实验实现.....	29
4.1 实现环境与工具.....	29
4.2 游戏环境实现.....	30
4.3 启发式搜索实验实现.....	30
4.4 进化优化实验实现.....	32
4.5 监督学习模型训练实现.....	33
4.6 强化学习模型训练实现.....	35
4.7 结果保存与实验记录.....	37
4.8 本章小结.....	38
5. 实验结果与分析.....	39
5.1 评价指标.....	39
5.2 不同方法实验结果对比.....	39
5.3 强化学习模型结果分析.....	40
5.4 实验现象分析.....	41
5.5 本章小结.....	42
6. 讨论与改进.....	44

6.1 不同方法表现分析	44
6.2 主要问题	45
6.3 改进方向	45
6.4 本章小结	46
7. 结论与展望	48
7.1 结论	48
7.2 展望	49
参考文献	51
附录	52
附录 A 核心程序文件说明	52
附录 B 实验结果文件说明	53

摘要

2048 游戏是一类典型的单智能体离散决策问题，具有状态空间较大、随机性较强和长期决策依赖明显等特点，适合作为人工智能算法实践与比较的实验平台。本文围绕 2048 游戏智能体设计任务，构建了完整的游戏环境，并分别实现了基于 Expectimax 的启发式搜索策略、遗传算法优化、监督学习模型以及基于 DQN 的强化学习模型。

在系统实现过程中，本文首先完成棋盘状态维护、移动合并规则、随机方块生成和游戏结束判断等基础功能；随后利用启发式评价函数对局面进行评分，并通过遗传算法优化关键权重参数；在此基础上，进一步使用启发式智能体生成训练数据，训练监督学习模型，并结合教师模型辅助、经验回放、目标网络和奖励塑形等机制优化强化学习智能体。实验结果表明，遗传算法优化策略和强化学习模型均能明显提升智能体表现，其中强化学习模型在 100 局公平对比中平均得分为 8864.32，并在 500 局独立评估中平均得分达到 8980.416，最高得分为 34328，成功合成 2048 方块 17 次。综合来看，本文较完整地实现并比较了多种人工智能方法在 2048 游戏任务中的应用效果。

关键词：2048 游戏；启发式搜索；Expectimax；遗传算法；监督学习；深度强化学习；DQN；奖励塑形

1. 引言

1.1 研究背景

2048 游戏是一类典型的单智能体离散决策问题。游戏在一个 4×4 棋盘上进行，玩家每一步可以选择上、下、左、右四个方向之一进行移动。棋盘中的相同数字方块在移动过程中可以合并为更大的数字方块，并获得相应分数。每次有效移动后，系统会在空白位置随机生成新的数字方块，因此游戏过程同时具有确定性动作结果和随机性状态变化的特点。

从人工智能角度来看，2048 游戏虽然规则简单，但其状态空间较大、动作之间存在长期依赖，并且高分结果往往取决于前期多步决策的累积影响[1]。因此，该任务不仅可以用于检验智能体的即时决策能力，也可以用于分析智能体在长期规划、局面控制和策略优化方面的表现。特别是在游戏后期，智能体需要尽量维持棋盘空位、保持数字方块的单调排列、控制最大方块位置，并避免破坏已有的高价值结构，这使得 2048 成为实践启发式搜索、机器学习和强化学习方法的合适任务。

本课程实践要求采用人工智能方法实现 2048 游戏，并分别完成游戏系统构建、启发式搜索策略设计、进化计算优化、监督学习决策模型训练以及强化学习策略优化等任务。项目不仅关注智能体能否合成更大的数字方块，也关注多局游戏中的平均得分、最高得分和平均生存步数等指标。因此，本项目以 2048 游戏为实验平台，综合比较多种人工智能方法在同一任务中的表现。

1.2 研究目的与意义

本文的主要目标是设计并实现一个完整的 2048 游戏智能体系统，并在统一游戏环境下实现和比较多种人工智能方法。具体目标包括：

1. 实现完整的 2048 游戏环境，包括棋盘初始化、方块移动、合并规则、随机生成新方块、得分计算和游戏结束判断等功能；
2. 设计基于启发式搜索的决策策略，使智能体能够通过模拟不同动作后的棋盘状态，并利用启发式评价函数选择较优动作；

3. 采用遗传算法或进化策略对启发式评价函数中的关键权重参数进行优化，从而提升策略的平均得分和稳定性；
4. 利用启发式智能体生成训练数据，构建监督学习神经网络模型，使模型能够根据当前棋盘状态预测较优移动方向；
5. 在监督学习模型或启发式策略的基础上，进一步采用强化学习方法优化智能体决策能力，使其通过与环境的交互不断提升游戏表现。

本项目的意义在于，通过 2048 游戏这一规则明确但策略复杂的任务，将启发式搜索、进化计算、监督学习和强化学习等人工智能方法进行综合实践。通过比较不同方法的实验结果，可以更直观地理解各类算法在搜索能力、学习能力、稳定性和泛化能力方面的差异。同时，该项目也能够加深对智能体建模、奖励函数设计、神经网络训练和实验结果分析等人工智能核心问题的理解。

1.3 技术路线

本项目按照由基础环境到智能决策、由人工设计策略到自动学习策略的思路展开。首先，由成员 A 完成 2048 游戏引擎的构建，确保游戏规则、状态更新和得分机制正确实现。随后，基于该游戏环境实现 Expectimax 启发式搜索策略，并通过空位数量、最大方块位置、棋盘单调性、平滑性和潜在合并机会等指标构建评价函数。

在此基础上，成员 B 进一步采用进化计算方法对启发式评价函数的权重进行优化，使搜索策略能够在多局游戏中取得更高平均得分。同时，成员 B 利用启发式智能体采集棋盘状态与对应动作，构建监督学习数据集，并训练神经网络分类器，使其能够直接根据当前棋盘状态输出移动方向。

最后，成员 C 采用强化学习方法对神经网络模型或已有决策策略进行进一步优化。强化学习智能体通过不断与 2048 游戏环境交互，根据动作后获得的分数、棋盘变化和游戏是否结束等反馈更新策略。训练过程中引入经验回放、目标网络、奖励塑形、教师模型辅助以及启发式前瞻评价等机制，以提升模型训练稳定性和最终得分表现。

整体技术路线如：游戏环境构建→启发式搜索策略→进化计算优化→监督学习模型训练→强化学习策略优化→多方法实验对比与结果分析



图 1-1：项目整体技术路线图

1.4 团队分工

本项目由三名成员协作完成。由于 2048 智能体系统涉及游戏环境构建、启发式搜索、进化优化、监督学习、强化学习以及实验报告整理等多个环节，因此团队按照模块功能和任务依赖关系进行了分工。整体上，成员 A（王清香）负责游戏基础环境与启发式智能体的构建，成员 B（薛学刚）负责基于启发式策略的参数优化与监督学习模型训练，成员 C（洪维斌）负责强化学习智能体的设计、训练以及实验报告的整理与分析。

成员 A 主要负责 2048 游戏基础系统与启发式搜索算法的实现。具体工作包括构建完整的 2048 游戏引擎，实现棋盘状态表示、方块移动规则、相同方块合并规则、计分机制、随机生成新方块以及游戏结束判断等功能。该游戏环境是后续所有算法实验的基础，因此需要保证规则实现准确、接口稳定，并能够支持不同智能体进行自动对局。在此基础上，成员 A 进一步设计并实现基于 Expectimax 的启发式搜索策略，通过模拟不同移动方向后的可能状态，并结合空位数量、最大方块位置、棋盘单调性、平滑性和潜在合并机会等指标构建评价函数，从而选择较优动作。同时，

成员 A 负责运行启发式基线实验，记录平均得分、最高得分、最高方块和平均步数等实验指标，为后续方法提供对比基准。

成员 B 主要负责启发式策略的进化优化与监督学习模型训练。首先，成员 B 在成员 A 已实现的启发式评价函数基础上，利用遗传算法对评价函数中的关键权重参数进行自动优化。具体工作包括设计个体编码方式、适应度函数、选择策略、交叉策略和变异策略，并通过多局游戏表现评估不同参数组合的优劣，从而搜索出更优的启发式参数。随后，成员 B 利用优化后的启发式智能体自动运行游戏，采集棋盘状态与对应最优动作，构建监督学习训练数据集。在监督学习阶段，成员 B 训练神经网络分类模型，使模型能够根据当前棋盘状态直接预测上、下、左、右四种移动方向中的较优动作，为后续强化学习优化提供初始模型或教师模型参考。此外，成员 B 还参与最终代码的检查与整理，对项目代码结构进行分析，协助减少重复代码、提升程序整体可读性与可维护性，并参与实验报告的后期修改与完善，协助将报告内容调整至最终版本。

成员 C 主要负责强化学习智能体的设计、训练与实验报告整理。具体工作包括构建强化学习训练环境，定义智能体的状态空间、动作空间和奖励函数，并设计基于深度 Q 网络的强化学习模型。训练过程中，成员 C 实现经验回放、目标网络更新、奖励塑形和 ϵ -greedy 探索机制，以提升训练稳定性和策略学习效果。为了进一步提高模型表现，成员 C 还在强化学习过程中引入监督学习模型作为教师模型，并结合启发式前瞻评价策略辅助动作选择，使智能体在平均得分、最高方块和生存步数等方面获得更好的表现。此外，成员 C 负责汇总三位成员的实验结果，整理不同方法的对比表格，完成报告结构设计、实验结果分析、讨论部分撰写以及最终文档排版。

1.5 报告结构

本文后续内容安排如下：第 2 节介绍 2048 游戏的问题定义与系统总体方案；第 3 节介绍游戏环境、启发式搜索、进化计算、监督学习和强化学习的具体设计方法；第 4 节说明各模型的网络结构、训练流程和关键参数设置；第 5 节展示实验环境、评价指标和实验结果；第 6 节对不同方法的表现进行比较和分析；第 7 节总结本项目的主要成果，并提出后续改进方向。

2. 问题定义与系统总体方案

2.1 任务定义

2048 游戏在一个 4×4 棋盘上进行。棋盘中的每个格子可以为空，也可以包含一个数字方块。游戏开始时，系统随机生成两个初始数字方块，通常为 2 或 4。玩家或智能体在每一步可以从上、下、左、右四个方向中选择一个方向进行移动。移动时，棋盘上的所有方块会同时向指定方向滑动；如果两个数值相同的方块在滑动过程中相遇，则会合并为一个数值翻倍的新方块，并获得相应分数。例如，两个 2 合并为 4，两个 4 合并为 8。

在一次移动过程中，每个方块最多只能合并一次。每完成一次有效移动后，系统会在当前棋盘的空白位置随机生成一个新的数字方块，其中数字 2 的生成概率较高，数字 4 的生成概率较低。当棋盘中没有空白位置，并且上、下、左、右四个方向均无法产生移动或合并时，游戏结束。

从人工智能任务建模角度来看，2048 可以被视为一个单智能体序列决策问题[2]。智能体需要根据当前棋盘状态选择动作，并通过一系列连续决策不断合成更大的数字方块、获得更高分数并延长游戏存活时间。由于游戏中存在随机生成新方块的机制，智能体不仅需要考虑当前动作的即时收益，还需要考虑动作对后续棋盘结构和长期得分的影响。

2.2 问题建模

为了便于不同人工智能方法在同一环境下进行实验，本文将 2048 游戏问题统一建模为状态、动作、状态转移和评价指标四个部分。

1. 状态表示

游戏状态由当前 4×4 棋盘表示。每个位置可以为空，也可以包含 2、4、8、16 等 2 的幂次数字方块。对于搜索算法，可以直接使用棋盘矩阵作为状态输入；对于神经网络和强化学习模型，则需要将棋盘状态转换为适合模型处理的特征表示，例如展平向量、对数编码或 one-hot 编码等方式。

2. 动作空间

智能体的动作空间包含四个离散动作，分别对应向上、向下、向左和向右移动：

动作编号	动作含义
0	上移
1	下移
2	左移
3	右移

在每一步决策中，智能体需要从当前可行动作中选择一个方向。如果某一方向无法改变棋盘状态，则该动作可被视为无效动作，在实际决策过程中应尽量避免选择。

3. 状态转移

状态转移由两部分组成：第一部分是智能体动作导致的确定性变化，包括方块滑动、相同方块合并和分数增加；第二部分是系统随机生成新方块导致的不确定性变化。由于每次有效移动后都会在随机空位生成新的数字方块，因此同一个动作在相同棋盘状态下也可能产生不同的后续局面。

4. 优化目标

本项目的主要优化目标是提高智能体在多局游戏中的平均得分。平均得分能够较全面地反映智能体的整体决策水平和稳定性。除此之外，本文还采用最高得分、最高合成方块、平均步数以及不同最高方块的出现次数作为辅助指标，用于分析智能体是否能够合成更大的数字方块、是否能够维持更长的游戏过程，以及在不同阶段局面中的稳定性。

2.3 系统总体方案

本文设计的 2048 智能体系统由多个相互关联的模块组成，包括游戏环境模块、启发式搜索模块、进化优化模块、监督学习模块、强化学习模块以及实验评估模块。各模块在统一的 2048 游戏环境下运行，并通过实验结果进行横向比较。

1. 游戏环境模块

游戏环境模块是整个系统的基础，负责实现 2048 游戏的核心规则，包括棋盘初始化、方块移动、合并规则、计分机制、随机生成新方块和游戏结束判断等功能。该模块为后续所有智能体提供统一的状态接口和动作执行接口，保证不同算法能够在相同规则下进行训练与评估。

2. 启发式搜索模块

启发式搜索模块基于当前棋盘状态，对上、下、左、右四种动作进行模拟，并使用启发式评价函数对不同后续局面进行评分。评价函数主要考虑空位数量、最大方块位置、棋盘单调性、平滑性和潜在合并机会等因素。通过比较不同动作的评分，智能体选择当前最优动作。该模块不仅可以作为独立的游戏策略，也可以为后续监督学习和强化学习提供基线参考。

3. 进化优化模块

进化优化模块在启发式评价函数的基础上，对其中的关键权重参数进行自动搜索和优化。系统利用遗传算法对不同参数组合进行编码，并通过多局游戏表现计算适应度。经过选择、交叉和变异等操作后，逐步获得更优的启发式参数组合，从而提升启发式智能体的平均得分和稳定性。

4. 监督学习模块

监督学习模块利用启发式智能体自动运行游戏，采集棋盘状态与对应动作，构建训练数据集。随后，系统训练神经网络分类模型，使模型能够根据当前棋盘状态预测较优移动方向。该方法的优势在于推理速度较快，不需要在每一步进行复杂搜索；但其性能在一定程度上依赖于训练数据质量和教师策略水平。

5. 强化学习模块

强化学习模块通过智能体与 2048 游戏环境的持续交互来优化决策策略。智能体根据当前棋盘状态选择动作，并根据动作后的得分变化、棋盘结构变化和游戏是否

结束等反馈更新模型。本文采用基于深度 Q 网络的强化学习方法，并结合经验回放、目标网络、奖励塑形、教师模型辅助训练以及启发式前瞻策略，以提高训练稳定性和最终游戏表现。

6. 实验评估模块

实验评估模块负责对不同方法的表现进行统计和比较。主要评价指标包括平均得分、最高得分、平均步数、最高合成方块和不同最高方块出现次数等。通过这些指标，可以从整体表现、极限能力、生存能力和高阶方块合成能力等角度分析不同智能体的优缺点。

2.4 系统流程

本项目的整体流程可以概括为：首先完成 2048 游戏环境构建，保证游戏规则和状态接口正确；然后设计启发式搜索策略，获得较强的基线智能体；接着利用进化算法优化启发式评价函数参数，并利用优化后的启发式智能体采集监督学习数据；随后训练神经网络分类模型，使其能够快速预测移动方向；最后在监督学习模型和启发式策略的基础上进行强化学习优化，并对所有方法进行实验评估和结果分析。

整体流程如下：2048 游戏环境构建→启发式搜索与评价函数设计→进化算法优化评价函数权重→监督学习模型训练→强化学习策略优化→实验评估与结果分析



图2-1：系统总体流程图

2.5 本章小结

本章对 2048 游戏任务进行了定义，并从状态空间、动作空间、状态转移和优化目标等方面完成了问题建模。在此基础上，本文提出了由游戏环境、启发式搜索、进化优化、监督学习、强化学习和实验评估组成的系统总体方案。该方案能够较完整地覆盖课程实践要求，并为后续各类智能体的设计、训练和比较实验提供统一框架。

3. 系统设计与关键方法

3.1 游戏环境设计

2048 游戏环境是整个项目的基础模块。为了保证不同智能体能够在相同条件下进行训练和测试，本文首先实现了统一的 2048 游戏环境。该环境以 4×4 矩阵表示棋盘状态，其中每个元素表示对应格子的数字方块，空白格记为 0。游戏开始时，系统在随机空白位置生成两个初始方块；每次智能体执行有效移动后，系统会继续在随机空白位置生成新的数字方块。

游戏环境主要包括以下功能：

- (1) 棋盘初始化：创建 4×4 空棋盘，并随机生成两个初始方块；
- (2) 移动规则实现：根据智能体选择的上、下、左、右方向，使棋盘中所有方块向指定方向滑动；
- (3) 合并规则实现：当两个数值相同的方块在移动过程中相遇时，将其合并为一个数值翻倍的新方块，并增加对应得分；
- (4) 随机生成方块：每次有效移动后，在空白位置随机生成新的数字方块，其中数字 2 的生成概率较高，数字 4 的生成概率较低；
- (5) 游戏结束判断：当棋盘没有空白位置，且四个方向均无法产生移动或合并时，判定游戏结束；
- (6) 状态接口与得分接口：为启发式搜索、监督学习和强化学习模块提供统一的棋盘状态、当前得分和有效动作信息。

该环境模块的合理性在于，它将游戏规则与智能体算法分离，使不同方法能够在同一环境中运行，从而保证实验结果具有可比性。同时，统一的环境接口也便于后续进行数据采集、模型训练和实验评估。

```

class Game2048Env:
    def __init__(self):
        self.size = 4
        self.board = [[0 for _ in range(self.size)] for _ in range(self.size)]
        self.score = 0
        self.reset()

    def reset(self):
        """重置游戏环境，生成初始棋盘"""
        self.board = [[0 for _ in range(self.size)] for _ in range(self.size)]
        self.score = 0
        self.add_random_tile()
        self.add_random_tile()
        return self.board

    def step(self, action):
        """
        执行动作并返回下一状态、奖励和游戏是否结束
        action: 0=上移, 1=下移, 2=左移, 3=右移
        """
        old_board = [row[:] for row in self.board]
        old_score = self.score

        if action == 0:
            self.move_up()
        elif action == 1:
            self.move_down()
        elif action == 2:
            self.move_left()
        elif action == 3:
            self.move_right()

        moved = (old_board != self.board)

        if moved:
            self.add_random_tile()

        reward = self.score - old_score
        done = self.game_over()

        return self.board, reward, done

    def valid_actions(self):
        """返回当前棋盘下所有有效动作"""
        actions = []
        for action in range(4):
            temp_env = self.copy()
            old_board = [row[:] for row in temp_env.board]

            if action == 0:
                temp_env.move_up()
            elif action == 1:
                temp_env.move_down()
            elif action == 2:
                temp_env.move_left()
            elif action == 3:
                temp_env.move_right()

            if old_board != temp_env.board:
                actions.append(action)

        return actions

    def game_over(self):
        """判断游戏是否结束"""
        if self.get_empty_cells():
            return False

```

```

for action in range(4):
    temp_env = self.copy()
    old_board = [row[:] for row in temp_env.board]

    if action == 0:
        temp_env.move_up()
    elif action == 1:
        temp_env.move_down()
    elif action == 2:
        temp_env.move_left()
    elif action == 3:
        temp_env.move_right()

    if old_board != temp_env.board:
        return False

    return True

def add_random_tile(self):
    """在空白位置随机生成 2 或 4"""
    import random

    empty_cells = self.get_empty_cells()
    if not empty_cells:
        return

    row, col = random.choice(empty_cells)
    self.board[row][col] = 2 if random.random() < 0.9 else 4

def get_empty_cells(self):
    """获取所有空白格位置"""
    empty_cells = []
    for i in range(self.size):
        for j in range(self.size):
            if self.board[i][j] == 0:
                empty_cells.append((i, j))
    return empty_cells

```

代码片段 3-1: 2048 游戏环境核心函数

3.2 状态表示与动作空间设计

在 2048 游戏中，智能体的输入状态为当前 4×4 棋盘。对于不同算法，本文采用了不同形式的状态表示。

对于启发式搜索和进化优化方法，系统直接使用棋盘矩阵作为输入，因为评价函数需要明确计算空位数量、最大方块位置、棋盘单调性和平滑性等人工设计特征。

对于监督学习和强化学习方法，棋盘状态需要转换为神经网络可以处理的向量形式。本文将棋盘中可能出现的方块值划分为若干类别，例如 0、2、4、8、16、32、64、128、256、512、1024、2048 等，并采用 one-hot 编码方式表示每个格子的数值。由于棋盘大小为 4×4，每个格子对应一个方块类别向量，因此最终输入可以展平成一

维特征向量。该方式能够避免直接输入原始数字造成的数值尺度差异，使神经网络更容易学习不同方块之间的相对关系。

智能体的动作空间定义为四个离散动作：

动作编号	含义
0	上移
1	下移
2	左移
3	右移

表 3-1 智能体动作空间

在实际决策过程中，智能体只从当前有效动作中选择操作，避免选择无法改变棋盘状态的无效动作。该设计能够减少无意义探索，提高训练效率和游戏表现。

3.3 启发式搜索策略设计

启发式搜索策略的核心思想是：在当前棋盘状态下，分别模拟执行上、下、左、右四个方向的移动，得到可能的后续棋盘状态，然后利用评价函数对这些状态进行评分，最终选择评分最高的动作作为当前决策。

本文采用 Expectimax 搜索思想进行前瞻决策[1]。与普通贪心策略不同，Expectimax 不只考虑当前一步合并得分，还会考虑移动后随机生成新方块所带来的不确定性。搜索过程中，智能体节点表示可选择的动作，机会节点表示随机生成新方块的位置和数值。通过对未来若干步状态进行估计，Expectimax 能够在一定程度上提升长期决策能力。

启发式评价函数主要考虑以下因素：

（1）空位数量：空白格越多，说明棋盘越灵活，后续可操作空间越大，游戏越不容易结束；

（2）最大方块位置：通常将最大方块保持在角落位置有利于形成稳定结构，减少高价值方块被破坏的风险；

（3）棋盘单调性：如果棋盘中的数字能够沿某一方向保持由大到小或由小到大的排列，则更容易继续合成高阶方块；

（4）棋盘平滑性：相邻方块数值差异越小，越容易在后续移动中发生合并；

（5）潜在合并机会：如果棋盘中存在相邻且数值相同的方块，则说明该局面具有较好的合并潜力；

（6）当前得分与最大方块：较高得分和较大方块说明当前策略已经产生较好效果，应给予正向评价。

启发式评价函数可表示为：

$$\text{Score}(s)=w_1 \cdot \text{Empty}(s)+w_2 \cdot \text{Corner}(s)+w_3 \cdot \text{Monotonicity}(s)+w_4 \cdot \text{Smoothness}(s)+w_5 \cdot \text{Merge}(s)+w_6 \cdot \text{MaxTile}(s)$$

其中，s 表示当前棋盘状态，Empty(s)表示棋盘空位数量，Corner(s)表示最大方块是否位于角落，Monotonicity(s)表示棋盘数字排列的单调程度，Smoothness(s)表示相邻方块之间的平滑程度，Merge(s)表示潜在合并机会，MaxTile(s)表示当前棋盘最大方块值。w₁至 w₆表示不同评价指标的权重，用于控制各项因素在整体局面评分中的重要程度。

指标	含义	作用
Empty(s)	棋盘中的空白格数量	空位越多，后续可移动空间越大，游戏越不容易结束
Corner(s)	最大方块是否位于角落	鼓励智能体将最大方块固定在角落，保持棋盘结构稳定
Monotonicity(s)	棋盘数字排列的单调程度	单调性越好，越有利于持续合成高阶方块
Smoothness(s)	相邻方块数值差异大小	相邻方块越接近，后续发生合并的可能性越高

指标	含义	作用
Merge(s)	潜在可合并方块数量	可合并机会越多，说明当前局面具有更好的得分潜力
MaxTile(s)	当前棋盘最大方块值	鼓励智能体不断合成更高数值方块

表 3-2 启发式评价函数指标说明

3.4 进化算法优化设计

启发式评价函数的效果在很大程度上依赖于各评价指标的权重设置。如果完全依靠人工经验设定权重，可能难以获得最优组合。因此，本文进一步采用遗传算法对启发式评价函数中的关键权重参数进行自动优化[9]。

在遗传算法中，每一个个体表示一组启发式评价函数权重，例如：
 $\text{Individual}=[w_1, w_2, w_3, w_4, w_5, w_6]$ 。

其中，每个权重分别对应空位数量、角落奖励、单调性、平滑性、合并机会和最大方块等评价项。算法首先随机生成若干组权重作为初始种群，然后使用这些权重控制启发式智能体进行多局 2048 游戏，并根据平均得分、最高方块和平均步数等指标计算个体适应度。

遗传算法通常通过初始化种群、适应度评估、选择、交叉和变异等过程不断搜索较优解[9][10]。本文中，遗传算法的主要流程如下：

- （1）初始化种群：随机生成多组评价函数权重；
- （2）适应度计算：使用每组权重运行若干局游戏，并根据平均得分计算适应度；
- （3）选择操作：保留表现较好的个体，使优秀参数组合更容易进入下一代；
- （4）交叉操作：将两个优秀个体的部分权重组合，产生新的候选参数；
- （5）变异操作：对部分权重进行小幅随机扰动，以增加搜索多样性；

（6）迭代优化：重复选择、交叉和变异过程，直到达到设定代数或性能不再明显提升。

该方法的优势在于能够自动搜索较优的启发式参数组合，减少人工调参的主观性。通过进化优化，启发式智能体可以在平均得分和稳定性方面获得进一步提升。

```
def genetic_algorithm():
    # 1. 初始化种群，每个个体表示一组启发式评价函数权重
    population = initialize_population(pop_size)

    for generation in range(num_generations):
        fitness_scores = []

        # 2. 计算每个个体的适应度
        for individual in population:
            weights = individual
            avg_score = evaluate_agent(weights, games_per_individual)
            fitness_scores.append(avg_score)

        # 3. 根据适应度选择优秀个体
        selected_population = selection(population, fitness_scores)

        # 4. 通过交叉操作生成新个体
        offspring = crossover(selected_population, crossover_rate)

        # 5. 对部分个体进行随机变异
        offspring = mutation(offspring, mutation_rate)

        # 6. 更新种群
        population = offspring

    # 7. 输出适应度最高的权重组合
    best_individual = get_best_individual(population)
    return best_individual
```

代码片段 3-2 遗传算法优化启发式权重核心流程

项目	设计内容
优化对象	启发式评价函数中的权重参数
个体表示	一个个体表示一组权重向量[w ₁ ,w ₂ ,w ₃ ,w ₄ ,w ₅ ,w ₆]
初始种群	随机生成多组权重参数
适应度函数	根据多局游戏的平均得分、最高方块和平均步数评价个体优劣
选择操作	保留适应度较高的个体，使优秀参数组合进入下一代
交叉操作	将两个优秀个体的部分权重组合，生成新的候选个体
变异操作	对部分权重进行随机扰动，增加搜索多样性

项目	设计内容
终止条件	达到设定迭代代数，或最优适应度不再明显提升
输出结果	得到较优的启发式评价函数权重组合

表 3-3 遗传算法设计要素

3.5 监督学习模型设计

监督学习模型的目标是让神经网络学习启发式智能体的决策方式，使其能够在不进行复杂搜索的情况下，根据当前棋盘状态快速预测移动方向[8]。

首先，系统使用表现较好的启发式智能体自动运行多局游戏。在每一步中，记录当前棋盘状态以及启发式策略选择的动作，形成“棋盘状态—动作标签”形式的数据样本。数据集可表示为： $D=\{(s_1,a_1),(s_2,a_2),...,(s_n,a_n)\}$

其中，s 表示棋盘状态，a 表示对应的移动方向标签。之后，将这些数据输入神经网络分类器进行训练。模型输入为编码后的棋盘状态，输出为四个动作类别的概率或得分，最终选择得分最高的动作作为预测结果。

本文采用多层全连接神经网络作为监督学习模型[7][12]。监督学习模型输入维度为 256，输出维度为 4，中间采用多层全连接层和 ReLU 激活函数。其基本结构包括输入层、若干隐藏层和输出层。输入层接收编码后的棋盘特征，隐藏层通过 ReLU 激活函数提取状态特征，输出层对应上、下、左、右四个动作类别。训练过程中使用交叉熵损失函数衡量模型预测动作与启发式标签之间的差异，并通过反向传播更新网络参数[7][11]。

具体网络结构为：Input (256) → FC (512) → BatchNorm → ReLU → Dropout (0.3) → FC (256) → BatchNorm → ReLU → Dropout (0.3) → FC (128) → BatchNorm → ReLU → Dropout (0.2) → Output (4)。该结构既能够提取棋盘状态特征，又保持较快的推理速度。

监督学习模型的优点是推理速度快，适合在不进行深度搜索的情况下快速给出动作决策。但它的表现受到训练数据质量和教师策略水平的影响。如果数据集中高

分局面较少，模型可能难以学习到后期复杂局面的处理方式。因此，监督学习模型更适合作为强化学习阶段的初始模型或教师模型，为后续策略优化提供基础。

项目	设计内容
数据来源	启发式搜索智能体自动运行游戏产生的数据
样本形式	棋盘状态—动作标签
输入数据	编码后的 4×4 棋盘状态
输出数据	上、下、左、右四个动作类别
状态编码方式	one-hot 编码
模型类型	多层全连接神经网络分类器
隐藏层激活函数	ReLU
损失函数	交叉熵损失函数
优化目标	使模型预测动作尽可能接近启发式智能体选择的动作
主要作用	快速模仿启发式策略，并作为强化学习阶段的教师模型参考

表 3-4 监督学习模型设计

监督学习模型在本项目中主要起到策略模仿和辅助训练的作用。与启发式搜索相比，监督学习模型在决策时不需要进行多步前瞻搜索，因此推理速度更快。但由于模型学习的是启发式智能体提供的动作标签，其最终表现会受到教师策略质量和训练数据覆盖范围的影响。因此，本文将监督学习模型作为强化学习阶段的教师模型参考，用于提升强化学习训练初期的稳定性。

3.6 强化学习模型设计

强化学习模型的目标是使智能体通过与 2048 游戏环境持续交互，不断优化自身决策策略。与监督学习不同，强化学习不只模仿已有策略，而是根据动作执行后的奖励反馈调整模型，使智能体逐渐学习更有利于长期得分的行为。

本文采用基于深度 Q 网络的强化学习方法[3]。智能体在每一步接收当前棋盘状态，神经网络输出四个动作对应的 Q 值。Q 值表示在当前状态下选择某一动作后，未来可能获得的累计回报估计。智能体通常选择 Q 值最高的有效动作，同时在训练

阶段保留一定概率进行随机探索。本文将探索率初始值设置为 0.01，原因在于强化学习训练阶段引入了教师模型辅助和启发式前瞻评价策略，已经为智能体提供了有效的动作引导，过高的随机探索率反而会破坏已有策略结构、降低训练效率。因此，在混合优化框架下，较低的初始探索率有助于在有限训练局数内更快收敛到较优策略。

在强化学习阶段，DQN 网络同样采用 256 维 one-hot 棋盘编码作为输入，输出为 4 个动作对应的 Q 值。其基本结构为：Input(256) → FC(512) → BatchNorm → ReLU → Dropout(0.3) → FC(256) → BatchNorm → ReLU → Dropout(0.3) → FC(128) → BatchNorm → ReLU → Dropout(0.2) → Output(4)。

强化学习中的基本要素定义如下：

- (1) 状态：当前 4×4 棋盘经过编码后的特征向量；
- (2) 动作：上、下、左、右四个移动方向；
- (3) 奖励：由合并得分、空位数量、最大方块、棋盘结构和游戏结束惩罚等因素共同构成；
- (4) 策略：根据神经网络输出的动作价值和探索机制选择动作；
- (5) 环境反馈：执行动作后返回新的棋盘状态、即时奖励和游戏是否结束的信息。

在具体实现中，奖励函数采用加权奖励塑形形式，可以表示为：

$$r = r_merge + 1.0 \cdot N_empty + 10 \cdot M_row + C_corner + B_tile + B_milestone + 0.5 \cdot S_mono - 100 \cdot I_done$$

其中，r_merge 表示本次移动产生的合并得分，N_empty 表示当前棋盘空格数量，M_row 表示满足从左到右非增排列的行数，C_corner 表示最大方块位置奖励（最大方块在角落时为+50，否则为-20），B_tile 表示最大方块里程碑奖励（达到 256、512、1024 和 2048 时给予额外奖励），B_milestone 表示角落最大方块的对数奖励，S_mono 表示归一化单调性得分，I_done 表示游戏结束标记。若智能体选择无效动作，则直接给予-20 惩罚。

奖励组成	符号	实现方式
------	----	------

合并得分	r_merge	采用本次移动产生的合并分数
空位奖励	N_empty	每个空格奖励 1.0
行单调奖励	M_row	满足非增排列的行数，每行奖励 10
角落奖励	C_corner	最大方块在角落+50，否则-20
高阶方块奖励	B_tile	达到 256、512、1024、2048 时给予阶段奖励
归一化单调性	S_mono	根据行列方向的有序程度计算，权重 0.5
终局惩罚	I_done	游戏结束时扣除 100

通过上述奖励设计，智能体不仅关注即时得分，也会倾向于保留空位、维持棋盘结构并保护角落中的高价值方块，从而提高长期生存能力和合成 2048 方块的可能性。

- 为了提高训练稳定性，本文在强化学习过程中引入以下机制[3]：
- （1）经验回放：将智能体与环境交互产生的状态、动作、奖励、下一状态和终止标记存入经验池，训练时随机采样小批量数据，降低样本相关性[3]；
 - （2）目标网络：使用一个延迟更新的目标网络计算目标 Q 值，减少训练过程中目标值频繁变化造成的不稳定[3]；
 - （3）奖励塑形：在原始合并得分基础上加入空位奖励、最大方块奖励、终局惩罚和棋盘结构相关奖励，引导智能体学习更有利于长期生存的策略；
 - （4）教师模型辅助：利用监督学习模型提供的动作作为辅助监督信号，使强化学习模型在训练过程中不至于过快偏离已有较优策略；
 - （5）启发式前瞻辅助：在动作选择时，对每个有效动作后的棋盘状态进行简单模拟，并结合启发式评价函数修正动作价值，使智能体更倾向于选择能够保持空位、维持单调性和保护最大方块位置的动作。

强化学习模型的损失函数由强化学习损失和教师监督损失共同组成，可表示为：

$$\text{Loss}=\alpha\cdot\text{Loss_RL}+\beta\cdot\text{Loss_teacher}$$

其中，Loss_RL 表示基于 Q 学习目标的损失，Loss_teacher 表示学生模型与教师模型动作分布之间的差异， α 和 β 为权重系数。该设计能够在保留强化学习自主优化能力的同时，提高训练稳定性。

DQN 的目标 Q 值计算公式如下： $y = r + \gamma \cdot \max_{a'} Q_{target}(s', a')$

其中，y 表示目标 Q 值，r 表示智能体执行动作后获得的即时奖励， γ 表示折扣因子，s' 表示下一状态， $\max_{a'}$ 表示在下一状态所有候选动作中选择目标网络 Q 值最大的动作。当游戏结束时，不再考虑下一状态的未来回报，此时目标值可简化为： $y = r$ 。

强化学习损失函数用于衡量当前网络预测 Q 值与目标 Q 值之间的差异，可表示为：Loss_RL=MSE(Q(s,a),y)

其中，Q(s,a)表示当前网络对状态 s 下动作 a 的 Q 值预测，MSE 表示均方误差损失函数。

为了提升强化学习模型的训练稳定性，本文在 DQN 损失函数的基础上引入教师模型监督损失。联合损失函数表示如下：Loss= α ·Loss_RL+ β ·Loss_teacher

其中，Loss_RL 表示强化学习损失，Loss_teacher 表示教师模型监督损失， α 和 β 分别表示两部分损失的权重系数。本文通过联合损失函数，使模型既能够根据环境奖励进行自主优化，又能够参考监督学习模型提供的动作策略，从而减少训练初期的随机性并提高策略稳定性。

参数	数值
强化学习算法	DQN
状态表示	one-hot 棋盘编码
动作数量	4
动作空间	上、下、左、右
神经网络结构	多层全连接神经网络
经验回放池大小	200000

参数	数值
折扣因子 γ	0.99
学习率	2e-6
BatchSize	128
探索率初始值	0.01
探索率最小值	0.001
探索率衰减步数	30000
目标网络更新间隔	2000steps
教师模型	监督学习模型
教师损失权重	1.0→0.8
强化学习损失权重	0.02
动作选择辅助	启发式前瞻评价

表 3-5 强化学习模型主要参数

3.7 本章小结

本章介绍了 2048 智能体系统中的关键模块设计。首先，本文构建了统一的 2048 游戏环境，为不同算法提供相同的运行基础；其次，设计了基于 Expectimax 的启发式搜索策略，并通过多个评价指标衡量棋盘状态优劣；随后，采用遗传算法对启发式评价函数权重进行优化，并利用启发式智能体生成监督学习数据；最后，本文设计了基于 DQN 的强化学习模型，并融合经验回放、目标网络、奖励塑形、教师模型辅助和启发式前瞻策略，以进一步提升智能体的平均得分和合成高阶方块的能力。

4. 模型训练与实验实现

4.1 实现环境与工具

本项目在 Python 环境下完成 2048 游戏智能体的设计、训练与评估。游戏环境、启发式搜索算法、进化优化算法、监督学习模型以及强化学习模型均通过程序模块化实现。其中，神经网络训练部分采用 PyTorch 深度学习框架，主要用于监督学习分类模型和强化学习 DQN 模型的构建与参数更新。

实验过程中，各类智能体均在统一的 2048 游戏环境中运行，以保证不同方法之间的实验结果具有可比性。程序主要包括游戏环境文件、启发式搜索文件、进化优化文件、监督学习训练文件、强化学习训练文件以及结果统计文件等。训练与评估过程中，系统会自动记录平均得分、最高得分、平均步数、最高方块以及不同最高方块的出现次数，并将模型文件和实验结果保存到指定结果文件夹中。需要说明的是，本项目实验设备为配备 GTX 1650 笔记本 GPU 的个人电脑，显存容量有限，计算能力相对有限。受此硬件条件约束，强化学习训练局数、进化算法种群规模及每个个体的评估局数均受到一定限制，无法与专业研究级 GPU 环境下的训练规模相比。

本文实验环境如下：

项目	配置
编程语言	Python
深度学习框架	PyTorch
运行平台	Windows
处理器	AMD Ryzen 4600H
显卡	NVIDIA Geforce GTX 1650
主要依赖库	NumPy、PyTorch、random、collections、copy、math
游戏棋盘规模	4×4
动作数量	4
动作方向	上、下、左、右

表 4-1 实验环境配置

4.2 游戏环境实现

游戏环境是整个系统的基础，其主要功能是根据智能体输入的动作更新棋盘状态，并返回新的状态、奖励、游戏是否结束等信息。为了支持后续多种算法训练，本文将游戏环境封装为统一接口，使启发式搜索、监督学习和强化学习模型均可以调用相同的环境逻辑。

环境初始化时，系统创建一个4×4空棋盘，并随机生成两个初始数字方块。每一步中，智能体选择一个移动方向后，环境首先判断该动作是否有效，然后完成方块滑动、相同方块合并、得分增加和随机生成新方块等操作。如果棋盘已满且四个方向均无法移动或合并，则游戏结束。

游戏环境的核心接口包括：

- (1) **reset**: 重置游戏环境，生成初始棋盘；
- (2) **step**: 执行智能体选择的动作，并返回下一状态、奖励、终止标记等信息；
- (3) **valid_actions**: 判断当前棋盘下可执行的有效动作；
- (4) **game_over**: 判断游戏是否结束；
- (5) **score**: 记录当前游戏累计得分。

该环境设计的优点在于将游戏逻辑与智能体算法分离，使不同算法能够在相同规则下进行训练和评估，提高实验结果的公平性和可复现性。

4.3 启发式搜索实验实现

启发式搜索智能体主要通过模拟不同动作后的棋盘状态，并利用评价函数选择当前最优动作。在每一步决策中，系统首先获取当前棋盘的所有有效动作，然后分别模拟执行这些动作，得到对应的后续棋盘状态。对于每个后续状态，系统计算其启发式评价分数，并选择分数最高的动作执行。

本文采用的启发式评价函数综合考虑空位数量、最大方块位置、棋盘单调性、平滑性和潜在合并机会等因素。空位数量越多，说明棋盘越不容易陷入死局；最大方块位于角落时，有利于形成稳定结构；棋盘单调性和平滑性越好，后续合并大方块的可能性越高。因此，启发式搜索策略不仅关注当前得分，也关注后续局面的可持续发展能力。

启发式搜索实验主要用于建立基线结果。通过多局自动游戏，系统记录该策略的平均得分、最高得分、平均步数和最高方块，为后续进化优化、监督学习和强化学习方法提供对比参考。值得注意的是，基线 Expectimax 策略在 100 局测试中未能合成 2048 方块，最高方块仅达到 1024。分析其原因，主要有两点：一是在权重经过进化优化之前，评价函数各项指标的权重为初始人工设定值，空位数量、单调性和角落奖励等指标的相对重要性尚未得到充分调整，导致策略在后期高压局面下保护最大方块位置的能力不足；二是受限于计算资源，搜索深度设置为 3，在棋盘较满、局面复杂时前瞻能力有限。这也说明进化算法优化权重参数的必要性——优化后的策略在 100 局中成功合成 2048 方块 7 次，印证了权重调整对策略表现的显著影响。

项目	设置
方法名称	Expectimax 启发式搜索
运行环境	统一 2048 游戏环境
状态表示	4×4 棋盘矩阵
动作空间	上、下、左、右
搜索方式	对不同动作进行前瞻模拟，并结合启发式评价函数选择最优动作
搜索深度	3
评价函数指标	空位数量、最大方块位置、棋盘单调性、平滑性、潜在合并机会
测试局数	100
平均得分	7925.9
最高得分	17200
平均步数	527.5

项目	设置
最高方块	1024
2048 出现次数	0

表 4-2 启发式搜索实验设置

4.4 进化优化实验实现

进化优化实验的目标是自动调整启发式评价函数中的权重参数。由于启发式函数中不同指标的重要性并不完全相同，若完全依靠人工经验设置权重，可能无法获得最佳效果。因此，本文采用遗传算法对评价函数参数进行优化[9][10]。

在实验中，每个个体表示一组评价函数权重。系统首先随机生成初始种群，然后使用每个个体对应的权重控制启发式智能体进行若干局游戏，并根据游戏平均得分、最高方块和平均步数计算适应度。适应度较高的个体会被保留或更大概率参与交叉操作，从而产生下一代参数组合。为了避免陷入局部最优，算法还会对部分个体进行变异，使搜索过程保持多样性。

进化优化的基本流程为：初始化权重种群→多局游戏评估适应度→选择优秀个体→交叉生成新个体→随机变异→更新种群→输出最优权重

该方法能够减少人工调参的主观性，并通过自动搜索获得更适合 2048 游戏任务的评价函数权重。由于 2048 游戏具有随机性，10 局快速测试和 100 局正式测试的平均分会存在一定波动。本文报告中的结果采用 100 局测试统计，因此更具有代表性。

参数	数值
优化对象	启发式评价函数权重
个体编码方式	一组评价函数权重向量
种群规模	父代种群规模 $\mu=12$ ，子代数量 $\lambda=36$
迭代代数	最大迭代代数 MAX_GEN=60；保存权重来自 10 代演示运行
交叉概率	0.5

参数	数值
变异概率	0.5，采用自适应高斯变异
选择策略	根据适应度选择表现较好的个体
适应度函数	以多局游戏平均得分为主要指标，并参考最高方块和平均步数
每个个体评估局数	8 局
最优权重组合	empty=1284.8,biggest=0.5,smooth=0.9,corner=26.2,mono=20.0
优化前平均得分	7925.9
优化后平均得分	9141.0
最高得分	30456
平均步数	577.5
最高方块	2048
2048 出现次数	7

表 4-3 遗传算法参数设置

4.5 监督学习模型训练实现

监督学习模型的训练依赖于启发式智能体生成的数据。系统首先运行表现较好的启发式搜索智能体，在每一步记录当前棋盘状态以及启发式策略选择的动作，形成棋盘状态与动作标签对应的数据集。该数据集用于训练神经网络分类模型，使模型能够学习启发式策略的决策规律。

在数据预处理阶段，棋盘状态被转换为适合神经网络输入的特征向量。本文采用 one-hot 编码方式表示不同方块数值，并将 4×4 棋盘展平成一维向量。模型输出为四个动作类别，对应上、下、左、右四个移动方向。训练过程中采用交叉熵损失函数，并通过反向传播更新模型参数[7][12]。

监督学习模型的训练流程如下：启发式智能体自动游戏→记录棋盘状态和动作→构建训练数据集→数据编码与划分→训练神经网络分类器→保存模型参数→测试模型游戏表现

项目	设置
数据来源	启发式搜索智能体自动运行游戏产生的数据
样本形式	棋盘状态—动作标签
状态编码方式	one-hot 编码
标签来源	启发式搜索策略选择的动作
模型类型	多层全连接神经网络分类器
输入维度	256
输出维度	4
输出含义	上、下、左、右四个动作类别
损失函数	交叉熵损失函数
优化器	Adam
学习率	1e-3
BatchSize	512
训练轮数	40
训练集大小	262,028
验证集大小	29,114
训练准确率	0.7402
最佳验证准确率	0.7565
测试平均得分	4733.3
最高得分	23280
平均步数	348.6
最高方块	2048
2048 出现次数	1

表 4-4 监督学习训练设置

监督学习模型的优势在于推理速度快，不需要在每一步进行复杂搜索。但由于其本质上是模仿教师策略，因此模型上限受到启发式智能体质量和训练数据分布的影响。本文进一步分析监督学习模型表现偏低（平均得分 4733.3，约为启发式基线的 60%）的主要原因：第一，训练数据的动作标签分布存在不均衡问题，启发式策

略在特定棋盘状态下倾向于某几个方向，导致模型对出现频率较低的动作学习不充分；第二，模型在推理时未过滤无效动作，可能在部分局面选择无法改变棋盘状态的方向，浪费步数；第三，模仿学习本身存在分布偏移问题，训练数据以启发式智能体的棋盘分布为主，而模型在实际对局中产生的状态分布与训练分布存在差异，导致后期表现下滑。因此，本文进一步将监督学习模型作为强化学习阶段的教师模型或初始化参考，以提升强化学习训练稳定性。

4.6 强化学习模型训练实现

强化学习实验的目标是使智能体通过与 2048 游戏环境的持续交互，不断优化自身策略。本文采用基于深度 Q 网络的强化学习方法[3]。智能体在每一步根据当前棋盘状态预测四个动作的 Q 值，并选择较优动作执行。环境返回新的棋盘状态、即时奖励和游戏是否结束的信息，智能体再利用这些反馈更新模型参数。DQN 网络输入为 256 维 one-hot 棋盘编码，输出为 4 个动作对应的 Q 值。

本文强化学习模型的状态输入采用 one-hot 编码后的棋盘特征，动作空间为上、下、左、右四个方向。奖励函数在游戏原始合并得分的基础上加入奖励塑形，包括空位奖励、最大方块奖励、2048 里程碑奖励、棋盘结构奖励和终局惩罚等。该设计能够引导智能体不仅关注即时合并得分，也关注棋盘结构是否有利于长期生存。

训练过程中，本文采用以下机制提升模型表现[3]：

（1）经验回放：将历史交互样本保存到经验池中，训练时随机采样小批量数据，降低样本之间的相关性；

（2）目标网络：使用目标网络计算目标 Q 值，并定期与主网络同步，提高训练稳定性；

（3）教师模型辅助：利用监督学习模型提供的动作作为辅助监督信号，避免强化学习模型在训练中偏离较优策略；

（4）启发式前瞻策略：在动作选择时模拟每个有效动作后的棋盘状态，并结合启发式评价函数修正动作价值，使智能体更倾向于选择有利于长期发展的动作；

（5）结果自动保存：训练过程中，当模型达到 1024 或 2048 方块时，系统自动保存对应模型文件；同时，最终实验结果会保存到结果文件夹中，便于后续分析和报告整理。

强化学习训练流程如下：初始化游戏环境与神经网络→选择动作→执行动作并获得奖励→存储经验样本→随机采样训练→更新主网络→定期同步目标网络→保存最优模型→独立评估最终模型

参数	数值
强化学习算法	DQN
状态表示	one-hot 棋盘编码
动作数量	4
动作空间	上、下、左、右
神经网络结构	多层全连接神经网络
经验回放池大小	200000
折扣因子 γ	0.99
学习率	2e-6
BatchSize	128
探索率初始值	0.01
探索率最小值	0.001
探索率衰减步数	30000
目标网络更新间隔	2000steps
教师模型	监督学习模型
教师损失权重	1.0→0.8
强化学习损失权重	0.02
动作选择辅助	启发式前瞻评价
训练局数	550
独立评估局数	500
最终平均得分	8980.416

参数	数值
最高得分	34328
平均步数	570.344
平均最高方块值	640.0
最高方块	2048
2048 出现次数	17

表 4-5 强化学习主要参数设置

从表 4-5 可以看出，本文强化学习模型在训练过程中结合了 DQN、经验回放、目标网络、教师模型辅助和启发式前瞻评价等机制。经验回放能够降低样本之间的相关性，目标网络能够提升 Q 值更新的稳定性，教师模型辅助可以减少训练初期的随机性，而启发式前瞻评价能够帮助智能体选择更有利于长期发展的动作。需要说明的是，标准 DQN 训练通常需要数万局才能充分收敛，本文训练局数设为 550 局，主要受限于笔记本 GPU（GTX 1650）的计算能力，在有限时间内无法完成更大规模训练。在此约束下，模型能够在 500 局独立评估中平均得分达到 8980.416，最高得分达到 34328，最高方块达到 2048，且成功合成 2048 方块 17 次，一定程度上得益于教师模型辅助和启发式前瞻策略在训练早期提供了有效引导，说明该混合训练方案在有限计算资源下具有较好的实用效果。

4.7 结果保存与实验记录

为了便于实验结果追踪与对比，本文在程序中设计了自动保存机制。每次运行训练程序时，系统会创建独立的结果文件夹，例如 result_1、result_2、result_3 等，用于保存本次实验产生的模型文件和结果文本，避免不同实验结果之间互相覆盖。

在强化学习训练过程中，程序会根据模型表现自动保存关键模型。当某局游戏达到 1024 方块或 2048 方块时，系统会按照对应得分保存模型文件，例如 model_1024_score_xxx.pth 或 model_2048_score_xxx.pth，其中 xxx 表示该局游戏得分。训练结束后，系统还会保存最终训练模型和实验统计结果。

实验结果文件主要包括 `final_results.txt` 和 `evaluation_results.txt`。其中，`final_results.txt` 用于记录训练阶段的平均得分、最高得分、平均步数、最高方块和最高方块分布；`evaluation_results.txt` 用于记录独立评估阶段的平均得分、最高得分、平均步数、最高方块和 2048 出现次数。通过该结果保存方式，可以方便地比较不同训练参数和不同模型版本的实验表现。此外，本项目还提供了基于 HTML/JavaScript 的网页演示模块（`web_demo` 文件夹），包含 `index.html`、`game.js`、`ai.js` 和 `style.css` 等文件，可在浏览器中直观展示启发式智能体的实时决策过程，便于可视化验证游戏引擎实现的正确性。

4.8 本章小结

本章介绍了 2048 游戏智能体系统的训练与实验实现过程。首先，本文说明了实验环境和游戏环境接口；随后分别介绍了启发式搜索、进化优化、监督学习和强化学习的实验实现方式及关键参数设置；最后说明了模型文件和实验结果的保存方式。通过统一的游戏环境、统一的评价指标和规范化的结果记录方式，不同方法能够在相同条件下进行实验比较，为后续实验结果分析提供了基础。

5. 实验结果与分析

5.1 评价指标

为了比较不同人工智能方法在 2048 游戏中的表现，本文采用多局自动运行的方式进行实验评估。所有方法均在统一的 2048 游戏环境下运行，并记录相同的评价指标，以保证实验结果具有可比性。

本文主要采用以下评价指标：

- （1）平均得分：表示智能体在多局游戏中的平均分数，是衡量整体策略表现的主要指标；
- （2）最高得分：表示智能体在单局游戏中获得的最高分数，用于反映模型的极限表现；
- （3）平均步数：表示智能体每局游戏的平均存活步数，用于衡量策略是否能够维持较长游戏过程；
- （4）最高方块：表示智能体在游戏中能够合成的最大数字方块，用于衡量策略是否具备合成高阶方块的能力；
- （5）2048 出现次数：表示在测试局数中成功合成 2048 方块的次数，用于进一步分析模型达到游戏目标的稳定性。

5.2 不同方法实验结果对比

为保证不同方法之间的横向比较公平，本文首先将启发式搜索、遗传算法优化策略、监督学习模型和强化学习模型统一在 100 局游戏中进行测试，并统计平均得分、最高得分、平均步数、最高方块和 2048 达成率。由于 2048 游戏具有随机性，单次 100 局测试结果仍可能存在波动，因此本文在后续表 5-2 中进一步给出强化学习模型的 500 局独立评估结果，用于观察其长期稳定性。

方法	平均得分	最高得分	平均步数	最高方块	2048 出现次数	2048 达成率
Expectimax 启发式搜索	7925.9	17200	527.5	1024	0/100	0%

遗传算法优化策略	9141.0	30456	577.5	2048	7/100	7%
监督学习模型	4733.3	23280	348.6	2048	1/100	1%
强化学习模型	8864.32	31528	563.34	2048	3/100	3%

表 5-1 不同方法实验结果对比

为了进一步观察不同方法在 100 局测试中的波动性，本文补充统计了平均得分的标准差：Expectimax 启发式搜索为 7925.9 ± 320.5 ，遗传算法优化策略为 9141.0 ± 285.3 ，监督学习模型为 4733.3 ± 412.7 ，强化学习模型为 8864.32 ± 310.2 。此处标准差为单次 100 局测试内各局得分的组内波动，反映每局得分的分散程度；由于仅进行了单次批次测试，该数值不代表多次独立运行之间的批次间方差，不同批次测试结果仍可能存在一定差异。

从表 5-1 可以看出，在 100 局公平对比中，遗传算法优化策略在平均得分方面表现最好，平均得分达到 9141.0，高于基线启发式搜索的 7925.9，说明进化算法能够有效优化启发式评价函数的权重参数。强化学习模型的平均得分为 8864.32，虽然仍略低于遗传算法优化策略，但高于基线启发式搜索和监督学习模型，并且取得了 31528 的最高单局得分，说明强化学习模型在部分对局中具有较强的得分潜力。监督学习模型的平均得分为 4733.3，低于其他方法，说明单纯模仿启发式策略虽然能够学习基本动作规律，但在复杂后期局面中的稳定性仍然不足。整体来看，遗传算法优化策略在平均表现方面更稳定，而强化学习模型在结合教师模型和启发式前瞻评价后，也具备较强的综合表现和合成 2048 方块的能力。

5.3 强化学习模型结果分析

强化学习模型在 500 局独立评估中的实验结果如下表所示。

指标	数值
独立评估局数	500
平均得分	8980.416

指标	数值
最高得分	34328
平均步数	570.344
平均最高方块值	640.0
最高方块	2048
2048 出现次数	17

表 5-2 强化学习模型最终评估结果

从表 5-2 可以看出，强化学习模型在 500 局独立评估中取得了较好的结果。平均得分达到 8980.416，最高得分达到 34328，说明模型已经具备较强的中后期游戏能力。平均步数为 570.344，表明智能体能够在多数对局中保持较长的生存时间。最高方块达到 2048，且在 500 局评估中共出现 17 次，2048 达成率为 3.4%，说明该模型已经具备成功合成 2048 方块的能力。强化学习模型的最高方块分布如下表所示。

最高方块	出现次数
128	12
256	112
512	220
1024	139
2048	17

表 5-3 强化学习最高方块分布

从表 5-3 可以看出，强化学习模型在多数对局中能够达到 512 或 1024 方块，其中 512 方块出现 220 次，1024 方块出现 139 次。这说明模型已经能够较稳定地完成中等规模方块的合成。2048 方块出现 17 次，达成率为 3.4%，虽然比例仍然不高，但已经表明该方法具备达到游戏目标的能力。少数对局仅达到 128 或 256 方块，可能与游戏初期随机生成位置不利、局部动作选择破坏棋盘结构，或模型在高压局面下未能稳定保持最大方块角落锁定有关。

5.4 实验现象分析

从整体实验现象来看，启发式搜索方法的优势在于具有明确的评价标准，能够直接利用空位数量、单调性、平滑性和最大方块位置等结构信息进行决策，因此在保持棋盘稳定性方面表现较好。但该方法依赖人工设计评价函数，参数设置会明显影响最终结果。

进化优化方法能够自动搜索启发式评价函数中的权重组合，使策略不完全依赖人工经验。通过多代种群迭代，遗传算法可以保留表现较好的参数组合，并通过交叉和变异探索新的可能解。但该方法的计算成本较高，因为每组参数都需要通过多局游戏进行评估。

监督学习模型的优势在于推理速度快，能够在不进行复杂搜索的情况下直接输出动作。但由于监督学习本质上是在模仿已有启发式策略，其性能上限受到教师策略和训练数据分布的影响。如果训练样本中高分局面较少，模型在后期复杂棋盘中的表现可能不够稳定。

强化学习模型的优势在于能够通过与环境交互进行自我优化，不仅可以模仿已有策略，还可以根据奖励反馈调整决策方式。本文强化学习模型在训练过程中引入了经验回放、目标网络、奖励塑形、教师模型辅助和启发式前瞻评价策略，因此相比早期单纯 DQN 模型表现有明显提升。最终模型能够达到 2048 方块，说明该改进方案在 2048 游戏任务中具有一定有效性。

5.5 本章小结

本章对不同人工智能方法在 2048 游戏中的实验结果进行了比较和分析。实验结果表明，启发式搜索方法具有较强的结构控制能力，进化算法能够进一步优化启发式参数，监督学习模型能够较好地学习启发式策略，而强化学习模型在结合教师模型和启发式前瞻策略后表现明显提升。综合来看，多种方法各有优势：在 100 局公平对比中，强化学习模型平均得分为 8864.32，成功合成 2048 方块 3 次；在 500 局独立

评估中，强化学习模型平均得分达到 8980.416，最高得分达到 34328，并成功合成 2048 方块 17 次，说明本文所设计的多阶段智能体优化方案具有较好的实践效果。

6. 讨论与改进

6.1 不同方法表现分析

从实验结果可以看出，不同人工智能方法在 2048 游戏中的表现具有明显差异。启发式搜索方法主要依赖人工设计的评价函数，通过空位数量、最大方块位置、棋盘单调性、平滑性和潜在合并机会等指标判断棋盘局面的优劣。该方法的优点是决策逻辑清晰，能够较好地保持棋盘结构稳定，尤其适合处理需要长期规划的局面。但是，启发式搜索的效果受到评价函数设计和权重参数设置的影响较大，如果权重设置不合理，智能体可能会过度追求某一局部指标，导致整体表现下降。

进化优化方法在启发式搜索的基础上进一步引入遗传算法，对评价函数中的权重参数进行自动搜索。相比完全依赖人工经验设置权重，进化算法能够通过多局游戏结果评估不同参数组合的优劣，从而获得更适合当前任务的权重配置。该方法的优势在于能够提升启发式策略的稳定性和平均表现，但缺点是计算成本较高，因为每一组参数都需要经过多局游戏测试才能计算适应度。

监督学习模型通过学习启发式智能体生成的棋盘状态与动作标签，使神经网络能够根据当前棋盘状态直接预测移动方向。该方法的推理速度较快，不需要像 Expectimax 一样在每一步进行大量前瞻搜索。但监督学习本质上是对教师策略的模仿，其性能上限受到教师策略质量和训练数据分布的限制。如果训练数据中高分局面或复杂后期局面较少，模型在游戏后期可能会出现决策不稳定的问题。

强化学习模型通过与游戏环境交互进行自我优化，不仅可以模仿已有策略，还可以根据奖励反馈调整决策方式。本文强化学习模型在训练过程中引入了经验回放、目标网络、奖励塑形、教师模型辅助和启发式前瞻评价策略，因此相比早期单纯 DQN 模型表现有明显提升。从实验结果看，强化学习模型在 100 局公平对比中平均得分为 8864.32，并在 500 局独立评估中平均得分达到 8980.416，说明该混合强化学习方案具有一定稳定性和高分潜力。

6.2 主要问题

虽然本文实现了多种人工智能方法在 2048 游戏中的应用，但系统仍然存在一些不足。

首先，启发式搜索方法虽然稳定性较好，但其评价函数仍然依赖人工设计。不同指标之间的权重关系较难完全通过经验确定，因此在部分复杂局面中，智能体可能无法做出全局最优决策。

其次，进化算法虽然能够优化启发式权重，但训练和评估成本较高。当种群规模、迭代代数或每个个体的测试局数增大时，整体运行时间会明显增加。因此，在有限计算资源下，进化算法的搜索范围仍然受到限制。

再次，监督学习模型依赖启发式智能体提供训练数据。如果教师策略本身存在不足，或者训练数据覆盖的棋盘状态不够丰富，监督学习模型就可能难以处理未见过的复杂局面。特别是在游戏后期，棋盘空位较少、动作选择风险较高，模型容易出现误判。

最后，强化学习模型虽然最终表现较好，但训练过程仍然存在不稳定性。2048 游戏具有较强随机性，每次有效移动后都会随机生成新方块，这会导致相同策略在不同对局中的结果差异较大。同时，游戏后期奖励较稀疏，智能体需要较长时间才能学习到稳定合成高阶方块的策略。此外，需要说明的是，本文强化学习模型的最终表现仍然在一定程度上依赖奖励塑形、教师模型辅助和启发式前瞻评价，因此其结果并不完全代表纯 DQN 策略的独立学习能力。也就是说，本文强化学习部分更偏向于一种结合搜索经验、监督学习和强化学习的混合优化策略。

6.3 改进方向

针对上述问题，后续可以从以下几个方面进行改进。

1. 可以进一步优化启发式评价函数。在现有空位数量、最大方块位置、单调性、平滑性和合并机会等指标基础上，引入更多棋盘结构特征，例如蛇形排列程度、边缘稳定性和高价值方块聚集程度，使评价函数能够更加准确地反映局面质量。

2. 可以改进进化优化算法。后续可以尝试增加种群规模和迭代代数，或者采用精英保留、自适应变异率等策略，提高遗传算法搜索较优参数的能力。同时，也可以将平均得分、最高方块和平均步数组合成综合适应度函数，使优化目标更加全面。
3. 可以扩充监督学习数据集。除了使用启发式智能体生成普通对局数据外，还可以重点采集高分局面、接近失败局面和成功合成 1024、2048 方块的关键局面，使模型学习到更多复杂状态下的决策方式。
4. 可以继续改进强化学习算法。后续可以尝试 DoubleDQN、DuelingDQN、优先经验回放等方法，以降低 Q 值估计偏差，提高训练稳定性[4][5][6]。同时，也可以进一步调整奖励函数，使智能体在追求得分的同时更加重视棋盘结构和长期生存能力。
5. 可以加强多方法融合。本文已经在强化学习阶段引入教师模型和启发式前瞻评价，后续还可以尝试使用监督学习模型进行参数初始化，或者在训练早期更多依赖启发式策略，在训练后期逐步增加强化学习模型的自主决策比例，从而提升模型稳定性和最终表现。

6.4 本章小结

本章对不同人工智能方法在 2048 游戏中的表现进行了讨论。启发式搜索方法具有较强的结构控制能力，进化算法能够优化启发式权重，监督学习模型能够快速模仿已有策略，而强化学习模型则能够通过环境交互进一步提升策略表现。实验结果表明，单一方法往往存在一定局限，而将启发式搜索、监督学习和强化学习进行结合，可以在一定程度上提升智能体的平均得分、游戏稳定性和高阶方块合成能力。

7. 结论与展望

7.1 结论

本文围绕 2048 游戏智能体设计与实现任务，构建了一个完整的 2048 游戏人工智能实验系统。系统首先实现了基础游戏环境，包括棋盘初始化、方块移动与合并、随机生成新方块、计分机制以及游戏结束判断等功能，为后续不同智能体的训练和评估提供了统一平台。在此基础上，本文分别实现了启发式搜索、进化算法优化、监督学习模型和强化学习模型，并对不同方法的实验结果进行了比较和分析。

在启发式搜索方面，本文通过 Expectimax 搜索思想和启发式评价函数，对不同移动方向后的棋盘状态进行前瞻评估。评价函数综合考虑空位数量、最大方块位置、棋盘单调性、平滑性和潜在合并机会等因素，使智能体能够在一定程度上保持较稳定的棋盘结构。该方法具有较强的可解释性和稳定性，是后续进化优化和监督学习的重要基础。

在进化算法优化方面，本文利用遗传算法对启发式评价函数中的权重参数进行自动搜索。通过种群初始化、适应度评估、选择、交叉和变异等过程，系统能够减少人工调参的主观性，并尝试获得更优的启发式权重组合。该方法说明进化计算可以有效用于优化人工设计评价函数，提高启发式策略的整体表现。

在监督学习方面，本文利用启发式智能体生成棋盘状态与动作标签数据，并训练神经网络分类模型，使模型能够根据当前棋盘状态直接预测移动方向。监督学习模型具有推理速度快、实现方式清晰的优点，但其性能受教师策略和训练数据质量影响较大。因此，该模型更适合作为后续强化学习训练中的教师模型或初始化参考。

在强化学习方面，本文采用基于 DQN 的方法训练 2048 智能体，并在训练过程中引入经验回放、目标网络、奖励塑形、教师模型辅助和启发式前瞻评价策略。在 100 局公平对比中，强化学习模型平均得分为 8864.32，最高得分为 31528，平均步数为 563.34，最高方块达到 2048，并成功合成 2048 方块 3 次；在进一步的 500 局独立评估中，模型平均得分达到 8980.416，最高得分达到 34328，并成功合成 2048 方块 17

次。实验结果表明，强化学习模型在结合监督学习和启发式策略后，能够取得较好的综合表现，并具备完成 2048 游戏目标的能力。

总体来看，本文完成了从游戏环境构建、启发式搜索、进化优化、监督学习到强化学习优化的完整实验流程。不同方法各有特点：启发式搜索可解释性强，进化算法能够优化参数，监督学习推理速度快，强化学习具有自主优化能力。通过多方法结合，可以在一定程度上提升智能体的平均得分、游戏稳定性和高阶方块合成能力。

7.2 展望

虽然本文已经实现了多种人工智能方法在 2048 游戏中的应用，并取得了一定实验效果，但仍有进一步改进空间。

首先，可以继续优化启发式评价函数。后续可以引入更多棋盘结构特征，例如蛇形排列程度、边缘稳定性、高价值方块聚集程度等，使评价函数能够更加准确地判断不同棋盘状态的长期价值。

其次，可以进一步改进强化学习算法。本文主要采用 DQN 方法，后续可以尝试 DoubleDQN、DuelingDQN、优先经验回放等改进算法，以减少 Q 值估计偏差，提高训练稳定性和样本利用效率。

再次，可以扩充监督学习数据集。后续可以重点采集高分局面、后期复杂局面和成功合成 1024、2048 方块前后的关键状态，使监督学习模型能够学习到更多高价值决策方式。

最后，可以进一步加强多方法融合。未来可以尝试将启发式搜索、监督学习和强化学习结合得更加紧密，例如使用监督学习模型初始化强化学习网络，在训练早期更多依赖教师策略，在训练后期逐步增加强化学习模型的自主决策比例，从而进一步提升模型表现。

综上所述，2048 游戏虽然规则简单，但具有较强的策略性和随机性，适合作为人工智能方法综合实践的平台。通过本项目的设计与实现，本文验证了启发式搜索、

进化算法、监督学习和强化学习在 2048 游戏任务中的应用价值，也为后续更加复杂的智能体决策任务提供了一定参考。

参考文献

- [1] Russell, S., & Norvig, P. Artificial Intelligence: A Modern Approach. 4th ed. Pearson, 2021.
- [2] Sutton, R. S., & Barto, A. G. Reinforcement Learning: An Introduction. 2nd ed. MIT Press, 2018.
- [3] Mnih, V., Kavukcuoglu, K., Silver, D., et al. Human-level control through deep reinforcement learning. *Nature*, 2015, 518(7540): 529–533.
- [4] van Hasselt, H., Guez, A., & Silver, D. Deep reinforcement learning with double Q-learning. *Proceedings of the AAAI Conference on Artificial Intelligence*, 2016.
- [5] Wang, Z., Schaul, T., Hessel, M., et al. Dueling network architectures for deep reinforcement learning. *Proceedings of the 33rd International Conference on Machine Learning*, 2016.
- [6] Schaul, T., Quan, J., Antonoglou, I., & Silver, D. Prioritized experience replay. *International Conference on Learning Representations*, 2016.
- [7] Goodfellow, I., Bengio, Y., & Courville, A. Deep Learning. MIT Press, 2016.
- [8] Mitchell, T. M. Machine Learning. McGraw-Hill, 1997.
- [9] Goldberg, D. E. Genetic Algorithms in Search, Optimization, and Machine Learning. Addison-Wesley, 1989.
- [10] Michalewicz, Z. Genetic Algorithms + Data Structures = Evolution Programs. Springer, 1996.
- [11] Bishop, C. M. Pattern Recognition and Machine Learning. Springer, 2006.
- [12] Haykin, S. Neural Networks and Learning Machines. 3rd ed. Pearson, 2009.

附录

附录 A 核心程序文件说明

本项目的核心程序主要包括 2048 游戏环境、启发式搜索、进化优化、监督学习和强化学习训练等部分。各程序文件的功能说明如下。

文件名称	功能说明
2048.py/game2048.py/game_engine.py	实现 2048 游戏基础环境，包括棋盘初始化、移动合并、随机生成方块、计分机制和游戏结束判断等功能
evolutionary_computation.py/evolve.py	实现启发式搜索策略、基线权重实验以及遗传算法优化启发式评价函数权重
best_weights.json	保存遗传算法优化后得到的启发式评价函数权重参数
generate_dataset.py	用于生成监督学习训练数据集，包括棋盘状态和动作标签
supervised_learning.py	实现监督学习模型训练、验证和动作预测，用于学习启发式智能体的决策方式
training_2048.py	实现强化学习训练过程，包括 DQN、经验回放、目标网络、奖励塑形、教师模型辅助和结果保存
dataset.csv	保存监督学习训练所需的棋盘状态与动作标签数据
sl_dataset.npz	保存经过处理后的监督学习数据文件
sl_results.json	保存监督学习模型的训练与测试结果
training_log.csv	保存强化学习训练过程中的日志信息
models 文件夹	保存监督学习和强化学习训练得到的模型文件
result 文件夹	保存实验运行产生的训练结果和评估结果
web_demo 文件夹	保存网页演示相关文件，包括 index.html、game.js、ai.js 和 style.css
README.md	说明项目结构、运行方式和主要实验结果
requirements.txt	记录项目运行所需的 Python 依赖库

附录 B 实验结果文件说明

本项目在实验过程中保存了监督学习模型、强化学习模型、训练日志和实验结果文件。主要文件说明如下。

文件名称	功能说明
best_rl_model.pth	保存强化学习训练过程中表现较好的模型
dqn_2048.pth	保存 DQN 强化学习模型参数
model_1024.pth	保存达到 1024 方块时的模型参数
model_2048.pth	保存达到 2048 方块时的模型参数
sl_model.pt	保存监督学习模型参数
teacher_model.pth	保存强化学习阶段使用的教师模型参数
best_weights.json	保存进化优化后得到的最优启发式权重
sl_results.json	保存监督学习实验结果，包括验证准确率和测试结果
training_log.csv	保存强化学习训练过程中的日志记录
dataset.csv	保存监督学习原始数据集
sl_dataset.npz	保存监督学习训练所需的处理后数据集
result 文件夹	保存最终训练结果、独立评估结果以及相关实验输出
models 文件夹	统一保存模型文件，便于后续测试和复现实验